Workflows

How `workflows`, `workflow_steps`, and `patient_workflows` (and their satellite tables) model a patient's journey through an encounter at a clinic.

The two layers

There is a **catalog** layer that defines *what workflows exist* and a **per-patient** layer that records *which workflows a specific patient is going through right now*.

```
Catalog (one row per kind of workflow)          Per-patient (one row per
workflow per encounter)
──────────────────────────────────────          ──────────────────────────

workflows                                       patient_workflows
└── workflow_steps                              ├──
patient_workflows_started
                                                ├──
patient_workflow_steps_completed
                                                └──
patient_workflows_completed
```

The catalog tables are the SQL projection of the constants in shared/workflow.ts (`WORKFLOWS`, `WORKFLOW_STEPS`). They exist in the database so that other tables (events, procedures, the `workflow` enum) can FK / type-check against them, and so that downstream tooling can use SQL joins for ordering and SNOMED lookups.

The per-patient tables track what's happened for an individual patient inside one `patient_encounters` row.

Catalog tables

`workflows`

Defined in db/migrations/20230101134900_workflows.ts.

column	type	notes
workflow	`workflow` enum (PK)	one of `registration`, `triage`, `consultation`, `maternity`, `referral_placed`, `emergency_escalation`, `stabilization`, `prescription_refill`, `doctor_review`, `create_google_meet`
snomed_concept_id	bigint, unique	SNOMED concept that names this kind of workflow (e.g. `TRIAGE`, `PATIENT_REGISTRATION`)
order	int8, unique	global presentation order

Seeded by db/seed/defs/07_workflows.ts from `WORKFLOW_SNOMED_CONCEPTS` in shared/workflow.ts. The seed runs in place (`always_run: true`) so adding a new workflow to `WORKFLOWS` in TS + running `deno task db:seed load` is enough — no full rebuild needed. The seed also ensures the `workflow` Postgres enum has every value present (`ensureAllEnumValuesExist`).

`workflow_steps`

Same migration file.

column	type	notes
workflow_step	varchar(255), PK	composite key, e.g. `"triage:warning_signs"` (see `workflowStepKey`)
workflow	`workflow` enum	which workflow this step belongs to
step	varchar(255)	step name within the workflow, e.g. `"warning_signs"`
snomed_concept_id	bigint, nullable	optional SNOMED concept for the step (see `WORKFLOW_STEP_SNOMED_CONCEPTS`)
order	int8, unique	global ordering across *all* workflow steps

Unique constraint `one_step_per_workflow` on `(workflow, step)`. Seeded by db/seed/defs/08_workflow_steps.ts from `WORKFLOW_STEPS` in shared/workflow.ts.

The unique `order` column is invariant-checked by test/models/workflows.test.ts — the seed temporarily drops and re-adds the unique constraint so existing rows can be reordered.

Source of truth

shared/workflow.ts is authoritative. It defines:

- `WORKFLOWS` — the enum members
- `WORKFLOW_SNOMED_CONCEPTS` — workflow → SNOMED concept
- `WORKFLOW_STEPS` — workflow → ordered list of step names
- `WORKFLOW_STEP_SNOMED_CONCEPTS` — optional per-step SNOMED concept (currently filled in for `triage` and `referral_placed`)
- `workflowStepKey(workflow, step)` → `"workflow:step"` (the PK in `workflow_steps`)
- `workflowStepPath(workflow, step)` → `"/workflow/step"`
- `firstIncompleteStep`, `firstStep`, `lastStep`, `prettyStepName`
- `canPerform(organization_employment, workflow)` → which department the worker is in that lets them do this workflow
- `WORKFLOW_NAV_LINKS` — used to render the sidebar of steps

shared/departments.ts maps each workflow to the department(s) responsible for it via `WORKFLOW_DEPARTMENTS` and exposes `departmentResponsibleForWorkflow(department, workflow)`. `referral_placed` and `create_google_meet` are open to all departments; everything else is owned by one or two specific departments (e.g. `triage` → `Triage`, `consultation` → `Primary care`).

Per-patient tables

`patient_workflows`

Defined in db/migrations/20230101137080_patient_workflows.ts. A standard table (`id`, `created_at`, `updated_at`) with:

- `patient_encounter_id` → `patient_encounters.id` (cascade delete)
- `workflow` → `workflow` enum

A row here means "this patient encounter is *planned* to go through this workflow". Multiple workflows can be planned per encounter — e.g. a returning patient seeking treatment gets `triage` and `consultation` rows inserted at the same time (see db/models/patient_encounters.ts `insertSeekingTreatmentForRegisteredPatient`).

A workflow can also be added mid-encounter through `patient_workflows.insertOne` from db/models/patient_workflows.ts, called by `startWorkflow` in routes/app/organizations/[organization_id]/patients/[patient_id]/open_encounter/start-workflow.tsx when `planning: 'create_anew_every_time'` (e.g. routing a triaged patient to `referral_placed`).

`patient_workflows_started`

Defined in db/migrations/20230101137086_patient_workflow_steps_completed.ts. A standard table with:

- `patient_workflow_id` → `patient_workflows.id` (cascade)
- `patient_encounter_employee_id` → `patient_encounter_employees.id` (cascade)
- unique constraint `patient_workflows_started_once` on `(patient_workflow_id, patient_encounter_employee_id)`

A row records "this employee has, at some point, picked up this workflow for this patient." Inserted by `patient_workflows.start` in db/models/patient_workflows.ts, which also bumps `employment_presence` to mark the employee as at-work and with this patient. The unique constraint + `onConflict.doNothing` makes start idempotent for a given (workflow, employee).

The presence of any row here for a `patient_workflow_id` is what flips the workflow's status from `not started` to `incomplete`/`in progress` in db/models/patient_encounters.ts `asWorkflowStatus`.

`patient_workflow_steps_completed`

Same migration. Standard table with:

- `patient_workflow_id` → `patient_workflows.id` (cascade)
- `workflow_step` → `workflow_steps.workflow_step` (the composite key)
- unique constraint `patient_workflow_step_once` on `(patient_workflow_id, workflow_step)`

One row per completed step. Inserted by `patient_workflows.completedStep`, called by `completeStep`/`completeLastStep` in the encounter middleware (routes/app/organizations/[organization_id]/patients/[patient_id]/open_encounter/_middleware.tsx). `completeStep` also emits an `OpenEncounterWorkflowStepCompleted` event (events/handlers.ts).

`patient_workflows_completed`

Pointer table — its `id` *is* a `patient_workflows.id`. The presence of a row means the workflow is finished (all steps done). Inserted by `patient_workflows.completedWorkflow`, idempotent via `onConflict.doNothing`. Called from `completeLastStep` once the final step of a workflow is submitted.

The `patient_presence` connection

db/migrations/20230101137087_patient_presence.ts — one row per patient currently inside the clinic. Has `current_workflow` and `next_workflow` columns (the `workflow` enum) plus a check constraint:

```
(department_name = 'Waiting room' AND current_workflow IS NULL AND
next_workflow IS NOT NULL)
OR
(department_name != 'Waiting room' AND current_workflow IS NOT NULL)
```

So a patient is *either* in the waiting room awaiting some next workflow, *or* actively in some workflow inside a department.

`patient_presence` is what the OpenEncounter routes read to decide which workflow's step pages to render. See `workflowHandler` in routes/app/organizations/[organization_id]/patients/[patient_id]/open_encounter/_middleware.tsx and `redirectToFirstIncompleteStep` in routes/app/organizations/[organization_id]/patients/[patient_id]/open_encounter/index.tsx.

End-to-end flow

A walk through a typical "patient walks in seeking treatment, gets triaged, then consulted" case:

1. Registration (Reception)

A receptionist starts a new patient via `patient_registration.start` (db/models/patient_registration.ts), which delegates to `patient_new_encounters.create` (db/models/patient_new_encounters.ts). One `WITH ... INSERT` chain creates:

- `patients` row
- `patient_encounters` row
- `patient_workflows` row with `workflow = 'registration'`
- `patient_presence` with `department_name = 'Reception'`, `current_workflow = 'registration'`
- `patient_encounter_employees` linking the receptionist
- `employment_presence` marking the receptionist with this patient
- `patient_workflows_started` linking the receptionist to the registration workflow

The receptionist fills out each registration step (`personal` → `this_visit` → `primary_care` → `contacts` → `confirm_details` → `terms_and_conditions` → `route_patient`). Each submission

inserts a row in `patient_workflow_steps_completed`. The final step calls `completeLastStep`, which writes `patient_workflows_completed` for the registration row.

2. Patient becomes "seeking treatment"

When registration finishes, `patient_encounters.insertSeekingTreatmentForRegisteredPatient` (db/models/patient_encounters.ts) runs and:

- Inserts **two** `patient_workflows` rows: one for `triage`, one for `consultation`.
- Updates `patient_presence` to either `Triage` (if the current employee works there and a room is free) with `current_workflow = 'triage'`, or `Waiting room` with `next_workflow = 'triage'`.
- If routed straight into triage, also inserts the corresponding `patient_workflows_started`.

The `consultation` `patient_workflows` row exists from this moment but stays unstarted until someone calls `start-workflow` for it after triage.

3. Triage

A triage nurse opens the patient → `workflowHandler` middleware fetches the encounter (`patient_encounters.getById`), which in its base query hydrates all `patient_workflows` for the encounter along with their `steps_completed` and `seen_patient_encounter_employee_ids`. `asWorkflowStatus` reduces those into one `WorkflowStatus` per workflow (`'not started'` | `'incomplete'` | `'in progress'` | `'completed'`), discriminated by whether `patient_presence.current_workflow` matches and whether `patient_workflows_completed` exists.

Each step page submission calls `completeStep` / `completeAndProceedToNextStep` → `patient_workflows.completedStep` + emits `OpenEncounterWorkflowStepCompleted`.

The triage `route_patient` step is the last step; it decides what happens next:

- `await_consultation` → marks the last step complete (`completeLastStep` writes both `patient_workflow_steps_completed` and `patient_workflows_completed`), then sets `patient_presence` to the waiting room with `next_workflow = 'consultation'`.
- `refer`/`manage_and_refer` → completes the triage workflow + calls `startWorkflow(ctx, 'referral_placed', { planning: 'create_anew_every_time', ... })`, which inserts a new `patient_workflows` row for `referral_placed`, a `patient_workflows_started` row, and moves `patient_presence` into the new workflow.

4. Consultation

The patient sits in the waiting room. A primary-care doctor sees them in the waiting room list (rendered by db/models/waiting_room.ts, whose action button POSTs to `start-workflow` with `workflow = 'consultation'`).

`startWorkflow` (routes/.../open_encounter/start-workflow.tsx) with `planning: 'do_not_create_only_start_if_already_planned'` uses the *existing* consultation `patient_workflows` row (created back in step 2), inserts `patient_workflows_started`, and moves

`patient_presence` into `Primary care` with `current_workflow = 'consultation'`. The doctor walks through `chief_complaint` → … → `close_visit`. The final step closes the encounter via `patient_encounters.close`.

Status derivation

`WorkflowStatus` (types.ts ~line 1836) is a discriminated union of:

- `not started` — `steps_completed = []`, `seen_patient_encounter_employee_ids = []`
- `incomplete` — has been started by ≥1 employee but isn't the current focus
- `in progress` — currently the `patient_presence.current_workflow` and has employees on it
- `completed` — `patient_workflows_completed` row exists, all steps done

`asWorkflowStatus` in db/models/patient_encounters.ts asserts the invariants between these (e.g. "if completed, all `WORKFLOW_STEPS[workflow]` are in `steps_completed`"). When the catalog and per-patient data drift, these asserts fire — useful tripwires when changing `WORKFLOW_STEPS`.

If you remove a step from `WORKFLOW_STEPS`, the seed in db/seed/defs/08_workflow_steps.ts deletes the now-orphan `workflow_steps` row, which cascades nothing of its own but will FK-violate against any existing `patient_workflow_steps_completed` rows referencing it. In dev that's fine after a `db:rebuild`; in any future production setting, a migration would be needed.

UI surface

The step sidebar is rendered by components/library/sidebar/Steps.tsx using `WORKFLOW_NAV_LINKS[workflow]` and the `steps_completed` array — completed steps get a check icon, others get a dot.

The wrapper that decides which step page to render for a URL is `workflowStepFromUrl` in the open-encounter middleware: it parses the URL, validates `workflow` and `step`, and asserts they exist in `WORKFLOW_STEPS`.

Cheat sheet: adding a new step

1. Add the step to the appropriate array in `WORKFLOW_STEPS` in shared/workflow.ts.
2. Optionally add a SNOMED concept entry in `WORKFLOW_STEP_SNOMED_CONCEPTS`.
3. Create the route file at `routes/app/organizations/[organization_id]/patients/[patient_id]/open_encounter/<workflow>/<step>.tsx` (use `OpenEncounterWorkflowPage`).
4. Run `deno task db:seed load` — the always-run seed in db/seed/defs/08_workflow_steps.ts will insert/reorder rows in `workflow_steps`.

Cheat sheet: adding a new workflow

1. Add to `WORKFLOWS` in shared/workflow.ts.
2. Add SNOMED in `WORKFLOW_SNOMED_CONCEPTS`, steps in `WORKFLOW_STEPS`.
3. Add to `WORKFLOW_DEPARTMENTS` in shared/departments.ts (which department(s) own it).
4. Create the route directory under `open_encounter/`.

5. `deno task db:seed load` — the `workflow` Postgres enum is auto-expanded by `ensureAllEnumValuesExist`; new `workflows` and `workflow_steps` rows are inserted by the always-run seeds.